



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2 GROUP 24

TECHNICAL REPORT:

APACHE SPARK FOR DATA ENGINEERING

LECTURER: DR. ARYATI BINTI BAKRI

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
SAFIYA NURSYAHADAH BINTI MASNOOR	A23CS0176

1.0 Executive Summary

This report examines Apache Spark as the primary engine for large-scale data engineering, highlighting its ability to replace disk-bound systems like Hadoop MapReduce with high-speed, in-memory computation. By utilizing the Resilient Distributed Dataset (RDD), Spark provides a unified platform for batch processing, streaming, and machine learning.

The analysis identifies the Driver Program, Cluster Manager, and Executors as core architectural components enabling parallel execution via a Directed Acyclic Graph (DAG). While Spark offers significant performance gains, challenges such as shuffle bottlenecks, which can consume over 50% of execution time and high memory demands require proactive management also surface.

Key recommendations focus on systematic optimization, including tuning memory fractions to reduce garbage collection by 40%, leveraging broadcast joins, and adopting Adaptive Query Execution. Furthermore, integrating Spark with Apache Airflow and Snowpark ensures scalable, compliant workflows in regulated sectors like healthcare. By implementing these practices and moving toward cloud-native Kubernetes deployments, organizations can achieve up to a 60% reduction in job completion times and a 50% decrease in operational costs.

2.0 Introduction

Apache Spark has become the de-facto engine for large-scale data engineering because it unifies batch, streaming, and interactive workloads on a single distributed platform . Originating from the AMPLab project at UC Berkeley in 2009 and open-sourced in 2010, Spark introduced the RDD abstraction that enables fault-tolerant, in-memory computation, dramatically reducing the latency of iterative algorithms compared with Hadoop's disk-bound MapReduce (Salloum et al., 2016). Consequently, Apache Spark provides a robust, extensible foundation for modern data-engineering workflows, enabling organizations to process petabyte-scale datasets with high throughput, low latency, and reliable fault tolerance. This report examines Spark's architecture, core abstractions, and best-practice optimisation techniques to guide practitioners in building scalable, maintainable data pipelines.

3.0 Background / Literature Review

Apache Spark has become a cornerstone in modern data engineering, offering a unified framework for diverse workloads such as batch processing, streaming, machine learning, and

graph analysis. Its core abstraction, the RDD, enables scalable algorithms and efficient data sharing across computations (Salloum et al., 2016). Unlike Hadoop's disk-based MapReduce, Spark leverages in-memory processing to reduce reliance on distributed file systems, resulting in significant performance gains. This architecture supports iterative algorithms, interactive queries, and real-time streaming within a single engine, eliminating the need for multiple specialized systems (Salloum et al., 2016).

Performance comparisons consistently highlight Spark's superiority over Hadoop. Shaikh et al. (2019) emphasize Spark's hybrid framework, which supports both batch and stream processing. They note that Spark's in-memory computation allows it to outperform MapReduce, particularly in batch workloads where intermediate results are retained in memory rather than written to disk. This design makes Spark faster and more flexible, though it also introduces challenges such as increased memory demands and more complex job scheduling (Salloum et al., 2016). Beyond technical architecture, Spark has demonstrated wide applicability in business and finance. Almagribi and Putranto (2025) found Spark extensively used for sentiment and trend analysis on social media, especially Twitter, to understand market dynamics and consumer behavior. Its predictive capabilities extend to customer churn modeling, asset pricing, and fraud detection, with efficiency and cost-effectiveness driving adoption.

While Apache Spark introduces a more flexible and advanced DAG model compared to the traditional map/reduce paradigm, managing and scheduling Spark jobs can be more complex than with MapReduce. In addition, Spark tends to be more sensitive to data quality issues, since retrieving data from remote memory incurs higher costs than accessing it from remote disk storage (Salloum et al., 2016). Research suggests that alternatives like Storm may be preferable for ultra-fast stream processing (Shoro & Soomro, 2015). Future directions include applying Spark to advanced financial forecasting, credit risk analysis, and Anti-Money Laundering detection, as well as integrating Spark into Data Lakehouse architectures for improved compliance and scalability (Almagribi & Putranto, 2025).

4.0 Analysis and Discussion

Apache Spark's architecture relies on three core components which are the Driver Program, Cluster Manager, and Executors where the Driver translates user code into a DAG of parallel

stages while Executors carry out computation as long-lived processes that minimize startup overhead (Khuat, 2025). This is underpinned by the RDD, a read-only partitioned collection of records that enables efficient data sharing across diverse workloads that previously required entirely separate distributed systems (Salloum et al., 2016). Figure 1 below illustrates this high-level distributed architecture, showing how the Driver, Cluster Manager, Worker Nodes, and Executors interact during application execution.

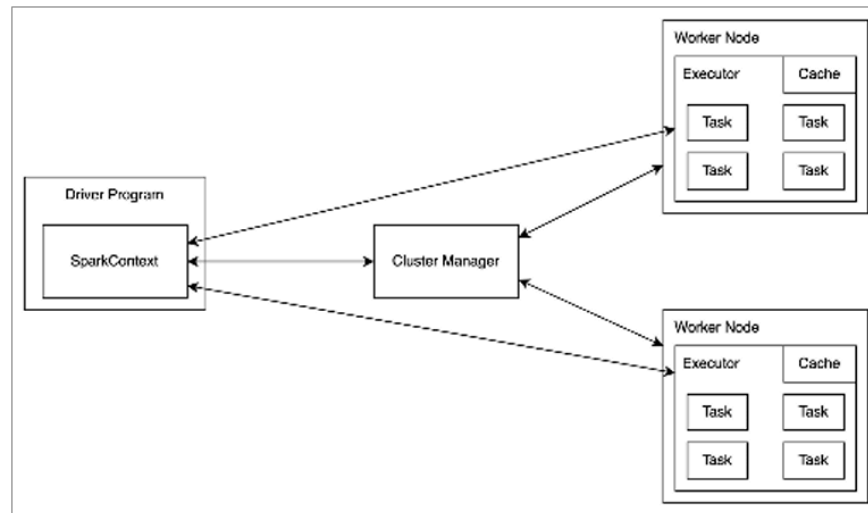


Figure 1 : Apache Spark Distributed Architecture (Khuat, 2025)

Through its in memory computing model, Spark significantly reduces the disk I/O bottleneck that constrains traditional MapReduce systems, making it significantly faster for iterative workloads (Dulam, 2025). Its unified programming model consolidates batch processing, streaming, machine learning, and graph analysis into a single engine, supporting rich APIs across Scala, Java, Python, and R, making it accessible to a broad range of developers while its MLlib library simplifies the design and implementation of scalable machine learning pipelines (Salloum et al., 2016). Real-world adoption reflects this versatility with a large online retailer reducing recommendation engine processing time by 40% through partitioning and caching optimizations, a global bank achieved sub-second response times for financial risk calculations through executor tuning, and a biotechnology company reduced genomic assembly processing from days to hours through memory management and shuffle optimization (Khuat, 2025). In healthcare, combining Spark with Apache Airflow and Snowpark enabled processing speeds of up to 1TB per hour with real-time analytics latency of under 2 seconds (Voola et al., 2022).

Despite these strengths, Apache Spark presents notable challenges. Managing and scheduling Spark jobs is considerably more complex than MapReduce due to its advanced DAG model, and Spark is more sensitive to data quality issues since retrieving data from remote memory is more costly than accessing it from remote disk storage (Salloum et al., 2016). Furthermore, for truly low-latency, high-throughput stream processing scenarios, Spark may not be the optimal choice unless Structured Streaming is proven efficient for the specific workload, with alternatives such as Apache Storm remaining preferable in certain real-time contexts (Salloum et al., 2016).

Memory management and shuffle optimization are the two most technically demanding performance areas. The unified memory model governs execution and storage memory through the `spark.memory.fraction` parameter, and optimal configurations reduce garbage collection time by up to 40%, while benchmarks show that over 50% of total execution time in many workloads is consumed by shuffle tasks alone (Dulam, 2025; Salloum et al., 2016). Across industries, systematic optimization has delivered 30–60% reductions in job completion times, 20–40% improvements in resource utilization, and 25–50% reductions in cloud computing costs, as summarized in Figure 2 below (Khuat, 2025).

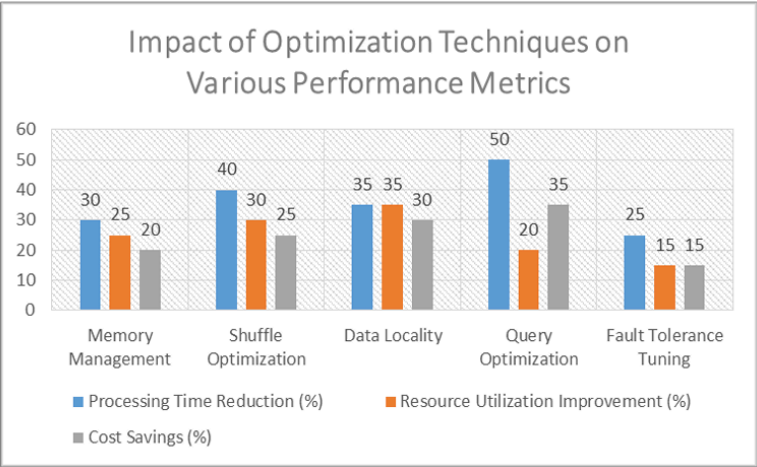


Figure 2 : Impact of Optimization Techniques on Various Performance Metrics (Khuat, 2025)

Looking ahead, Kubernetes integration, serverless Spark, and GPU acceleration are key emerging trends, with organizations that quickly adopt new Spark features demonstrating on average 30% greater processing efficiency compared to those that lag in adoption (Khuat, 2025).

5.0 Recommendations

Organizations should begin by systematically tuning memory configurations specifically `spark.executor.memory`, `spark.driver.memory`, and `spark.memory.fraction` based on actual workload profiling, with off-heap memory and Kryo serialization further reducing garbage collection overhead and data transfer costs. Automated tuning approaches have demonstrated job completion time improvements of 2 to 20 times over default configurations (Dulam, 2025). Shuffle optimization should be treated as a routine practice through adjusting `spark.sql.shuffle.partitions`, increasing `spark.shuffle.file.buffer`, using broadcast joins for smaller tables, and enabling Adaptive Query Execution for dynamic plan adjustments at runtime (Dulam, 2025; Khuat, 2025).

Teams should institutionalize the Spark Web UI as a proactive monitoring tool, regularly reviewing metrics such as task skew, shuffle sizes, and garbage collection time after each major job run to identify bottlenecks before they escalate (Khuat, 2025). Data locality should be a deliberate design consideration from the outset through thoughtful partitioning, selective caching, and broadcast variables for small reference datasets (Khuat, 2025). Finally, fault tolerance settings should be calibrated to workload critically rather than applied uniformly, and organizations should align their long-term infrastructure strategy with cloud-native deployments including Kubernetes-based orchestration and serverless Spark to ensure scalability as data demands grow (Khuat, 2025; Salloum et al., 2016).

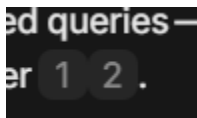
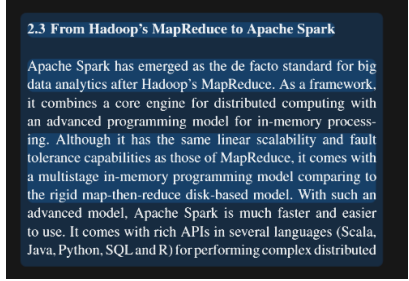
6.0 Conclusion

In conclusion, Apache Spark has proven to be a powerful and versatile framework for modern data engineering, delivering significant performance advantages over traditional systems through its in-memory computing model and unified architecture. When paired with systematic optimization practices and complementary tools such as Airflow and Snowpark, organizations across industries have achieved substantial gains in processing efficiency, resource utilization, and cost reduction.

References

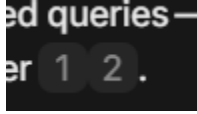
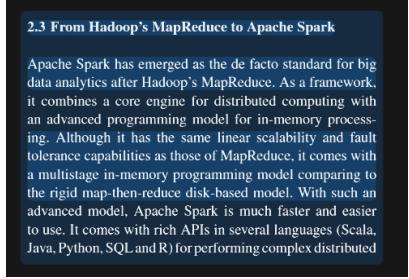
- Almagribi, A. B., & Putranto, B. P. D. (2025). Apache Spark for business and financial data engineering: A systematic literature review. *International Journal of Software Engineering and Computer Science*, 5(3), 1307–1321. <https://doi.org/10.35870/ijsecs.v5i3.5419>.
- Dulam, A. (2025). Enhancing data processing with Apache Spark: A technical deep dive. *World Journal of Advanced Engineering Technology and Sciences*, 15(03), 1279–1284. <https://doi.org/10.30574/wjaets.2025.15.3.0910>
- Khuat, Q. H. (2025). Mastering Apache Spark architecture: A guide to optimizing data processing workflows. *World Journal of Advanced Engineering Technology and Sciences*, 15(01), 910–923. <https://doi.org/10.30574/wjaets.2025.15.1.0294>
- Salloum, S., Dautov, R., Chen, X., Peng, P. X., & Huang, J. Z. (2016). Big data analytics on Apache Spark. *International Journal of Data Science and Analytics*, 1(3), 145–164. <https://doi.org/10.1007/s41060-016-0027->.
- Shaikh, E., Alufaisan, Y., Mohiuddin, I., & Nahvi, I. (2019). Apache Spark: A big data processing engine. In *2019 2nd IEEE Middle East and North Africa Communications Conference (MENACOMM)* (pp. 1–6). IEEE. <https://doi.org/10.1109/MENACOMM.2019.8703632>.
- Shoro, A. G., & Soomro, T. R. (2015). Big data analysis: Apache Spark perspective. *Global Journal of Computer Science and Technology: C Software & Data Engineering*, 15(1), 1–10.
- Voola, P. K., Murthy, P., Kumar, R., Goel, O., & Jain, A. (2022). Scalable data engineering solutions for healthcare: Best practices with Airflow, Snowpark, and Apache Spark. *International Journal of Computer Science and Engineering*, 11(2), 23–40.

Appendix / Logbook

Date and Time	Task	Details (Ex: AI Prompt, Verification Steps, Related links, etc)	Done By
4/4/2026 12:06 PM	Search the articles related to apache spark for data engineering in google scholar	Search prompt: apache spark for data engineering	Ika
4/4/2026 12:24 PM	Search the articles related to apache spark for data engineering in utm library database	Search prompt: apache spark for data engineering	Ika
4/4/2026 7:41 PM	Extract useful info from all the research papers found with the help of AI.	AI: Anara AI Prompt: can you extract useful info from these files that i can use for my literature review in my technical report titled Apache Spark for data engineering Verification step: Check if the citation put at the end of each point pointed out by AI matches the actual content of the papers it extracts the information from. Examples can be seen in the screenshot attached below:  	Ika

4/4/2026 8:27 PM	Ask Copilot to write a literature review based on the research papers found and compare the points to Anara AI.	AI: Copilot Prompt: using these research papers, can you write me a literature review with word count of 300-400 for my technical report titled Apache Spark for Data Engineering. Put the in-paragraph citations when needed. Verification steps: Recheck if the point made in the paragraphs written by AI really exist in the reference it cites.	Ika
4/4/2026 8:41 PM	Brings out all the important points pointed out by AI tools from research papers raw to another document to see, read and write the points easier.	-	Ika
5/4/2026 11.30 AM	Search the articles related to apache spark for data engineering in AnswerThis, ResearchRabbit and ConnectedPapers	Search prompt : Apache Spark for Data Engineering	Safiya
5/4/2026 12:24 PM	Write the literature review section.	AI: Copilot Prompt: in this pdf, I put the raw text from the paper I gave to you and the points that are relevant to the literature review. Can you combine what you had written with these points to form a complete literature review section with the word count of 300-400 words. Make sure to use in paragraph citation when needed and list out the reference in the end using APA format. Verification steps: Recheck the paragraphs written by AI and compare it to the raw text from research papers extracted	Ika

		in the pdf to make sure the point still stands and details are correctly aligned.	
5/4/2026 11:00 PM	Extract useful info from all the research papers found with the help of AI.	AI: Gemini Prompt: can you extract useful info from these files that i can use for my analysis and discussion, and recommendation in my technical report titled Apache Spark for data engineering Verification step: Check if the citation put at the end of each point pointed out by AI matches the actual content of the papers it extracts the information from.	Safiya
6/4/2026 10:30 AM	Ask Claude AI to write a literature review based on the research papers found and compare the points to Gemini.	AI: Claude AI Prompt: using these research papers, can you write me an analysis and discussion, and recommendation for my technical report titled Apache Spark for Data Engineering. Put the in-paragraph citations when needed. Verification steps: Recheck if the point made in the paragraphs written by AI really exist in the reference it cites.	Safiya
7/4/2026 11:00 PM	Write introduction	AI: Anara and Copilot Prompt: can you write an introduction section for my technical report of apache spark for data engineering using reliable sources Verification steps: Check if the citation put at the end of each point pointed out by AI matches the actual content of the papers it extracts the information from. Examples can be seen in the screenshot	Ika

		attached below:  	
8/4/2026 10:00 PM	Write Conclusion	AI: Claude AI Prompt: can you write an conclusion section for my technical report of apache spark for data engineering using reliable sources Verification steps: Check if the citation put at the end of each point pointed out by AI matches the actual content of the papers it extracts the information from.	Safiya
9/4/2026 12:23 AM	Write executive summary	AI: Gemini Prompt: Adjust the executive summary to Provide a concise summary (approximately 150–200 words) highlighting the paper's purpose, key findings or arguments, and main conclusions or recommendations. Verification steps: Report files are uploaded to AI. When it writes the intro, it cites the sources from the report, check this info to make sure it matches the actual content of the report.	Ika